

Technical Feasibility

October 23, 2026

Team Name TLATeX

Project ReconTEXTualization

Team Sponsor Chris Ortiz

Team Mentor Smirthya Iyer

Team Members Tyler Carver
Aron Gebrezghr
Daniel Schalesky
Evelyn Torres

Table of Contents

Introduction....3
Technical Challenges....5
Major Technical Challenges....5
Technical Analysis....6
Challenge One: Learning Java to Update the TLATeX Source Code....6
Challenge Two: Accounting for All Possible Syntax Combinations and Edge Cases....8
Challenge Three: Ensuring the Correctness of Math Notation....10
Challenge Four: Supporting Custom Colors....11
Challenge Five: Supporting Image Injection....14
Solution Integration....17
Conclusion....20

Introduction

TLA+ is a high-level, declarative formal specification language used for designing, modeling, and verifying dynamic systems. LaTeX is a free, open-source document preparation and typesetting system. TLATeX is a tool used by academics, researchers, and engineers to typeset TLA+ specifications into LaTeX. LaTeX can include images, colors, and mathematical notation, but TLATeX has not been updated to support those features. An update to the software is unavoidable because the current version of TLATeX does not support images, colors, or operators in its output files.

Currently, the problem is not being addressed. Users create LaTeX documents with TLA+ specifications and manually edit in the features they want.

The proposed solution is to modify the existing TLATeX source code so it can convert TLA+ specifications into LaTeX documents with pictures, colors, and mathematical symbols. Users will be able to check whether a TLA+ specification's logic is correct and turn it into a LaTeX document using commands in the same integrated development environment. The team will fork the existing code, develop the features in that repository, and submit a pull request to the tlaplus repository. The scope of this project is to add code to existing tools, not to change the TLA+ language.

The sponsor of the project is Chris Ortiz, a principal engineer at SanDisk. The sponsor anticipates greater adoption of TLA+ and its tools at SanDisk, where the tool will be used for documentation. The expected impact of the updated TLATeX is increased adoption of TLA+ in industry and academia.

Our sponsor has provided a declassified internal SanDisk memo that was formatted manually using LaTeX (Figure 1). This document will serve as a template to guide the development of TLATeX. Aligning the output of TLATeX with the SanDisk template will simplify the integration of TLA+ into SanDisk's development process.

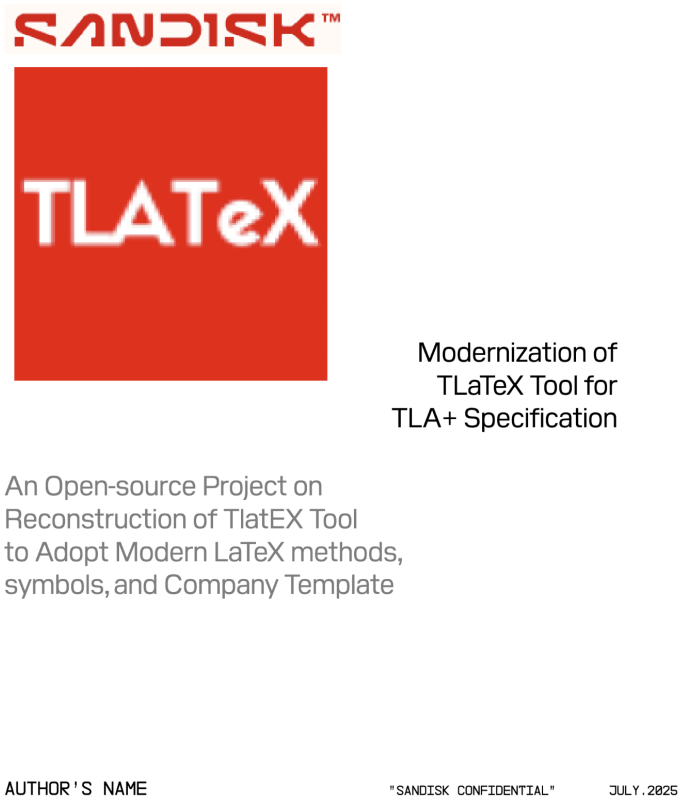


Figure 1: Cover page of the SanDisk documentation template provided by the sponsor

TLA+ has been used to catch logical errors in real systems. For example, during the development of the Xbox 360 in 2004, a TLA+ model was developed for a memory coherence protocol. The bug it found was reported to IBM, which confirmed that the bug existed and that its own tests could not find it. Had the bug not been discovered, the chips would have deadlocked after four hours of use, and the Xbox 360 would have missed its initial launch date. Amazon Web Services (AWS) has been using TLA+ since 2011. Its engineers have said that formal methods find bugs in system designs and allow engineers to create a system design and test it.

The goal of this technical feasibility document is to assess critical technical challenges and associated risks, investigate appropriate solutions, and justify the technologies and architecture decisions the team plans to pursue.

Technical Challenges

The functional requirements are that the TLATeX tool must be able to insert images, change the font color, and produce proper mathematical notation. The new features must not break the document being converted to LaTeX.

The non-functional requirements are that users must be able to make changes, such as changing character colors or adjusting images, using TLA+ formatting and tools in less than three minutes. Another requirement is contributing to the open-source project and successfully integrating our project code without introducing bugs.

Adding new features to the TLATeX tool will require changes to the tla2tex source code on how to handle new commands and convert TLA+ to a LaTeX document. The addition of the features have raised these following challenges:

Major Technical Challenges

1. Learning Java in order to update the source code
2. Accounting for all possible syntax combinations and edge cases
3. Ensuring the correctness of math notation
4. Supporting custom colors
5. Supporting image injection

Technical Analysis

Challenge One: Learning Java to Update the TLATeX Source Code

Challenge Description

The TLATeX source code is written in Java, so the team needs to understand the language and the existing source code before making updates. A core understanding of Java fundamentals and syntax will significantly improve the team's use of the language. Lacking a true foundational understanding of Java would hinder us greatly when we are tasked with serious changes to the source code. Since we use VS Code as our environment for changing TLATeX, we will need to access the Java code and know how to change it. The challenge is learning how the different parts of the program work together and finding where to make changes without causing problems.

Success Criteria

- We have a strong foundational understanding of Java code and syntax: everyone on the team is able to launch the environment, implement planned changes to the source code, then push those changes and see the results. Changes will be defined and agreed on before any work begins.
- We are able to identify and explain the Java code responsible for a planned change in our VS Code source environment, so that we not only make changes but also understand where and how they take place.
- We make changes to the source code that are meaningful enough to push to the main repository, and we understand how they are implemented. These are not basic changes such as names and titles, but extensive, impactful changes such as document color formatting, visual math notation, and other meaningful changes to the source code.
- We pass the relevant tests, such as visual confirmation and functionality checks, and confirm that existing output works without compilation errors. When parsed, the model works exactly as expected, with no surprises.

Candidate Approaches

- **Online courses and videos.** Use free online resources to solidify a basic understanding of Java, such as Udemy through our university account. Udemy offers in-depth lectures and tutorials for learning Java from the ground up. These can be paired with the abundant free resources on the internet, such as YouTube playlists like “Java Full Course for free” by Bro Code.
- **Learning by doing.** Learn while implementing small changes in the VS Code environment, allowing limited AI guidance, which will help us learn from our mistakes and find more efficient ways to implement source code changes. For example, when learning a new way to implement a change, team members can make multiple changes without pushing to the main repository, to avoid conflicts. This lets team members approach learning in an engaging way. Exploring the environment to see where files are placed and hosted will also give a more concrete understanding of the workflow.

- **Team walkthroughs.** Team members can work together to help each other understand the source code better. Our client, Chris Ortiz, will demonstrate code changes and their implementation in VS Code using the Microsoft Live Share extension during our team meetings. Because Chris has the most experience with the TLA+ language and the TLATeX software, he will be a great source of learning, and he will point us toward resources for figuring out how to make changes in Java. Although our allocated time is limited, the one-hour weekly meetings will allow time for this learning.
- **Team review before merging.** When source code changes are ready to be merged into the main repository, the exact changes will first be confirmed within the team, and the Java changes will be reviewed and cleared of any errors. Once the team reaches agreement, the changes will be pushed to the main repository shared with Chris Ortiz, which will host the essential project changes that reflect our goals.

Evaluation and Evidence

A basic-to-advanced understanding of Java will give us a foundation for approaching our capstone problem. It will certainly be a compounding effort of learning while implementing changes to the source code. When every team member is able to access their environment (VS Code), go into the source code, complete changes to formatting, colors, math notation, and so on, and then compile with no errors, that will be the evidence that we have met this goal and overcome this challenge. We are not expected to become Java professionals, but to be proficient enough to complete tasks and to research and overcome the challenges that will eventually arise.

Recommended Approach

We recommend learning Java while making small changes to our repository's source code, using Udemy courses and free Java resources from YouTube. Starting with simple changes, we will rebuild the program and check the output before moving on to larger changes.

Table 1: Comparison of approaches for learning Java

Criterion	Udemy Java Course Videos	YouTube Java Tutorial Videos	Chris Ortiz Team Meetings
Cost Effectiveness	High	High	High
User Convenience	High	High	Medium
Reliability	High	High	Medium

The availability and reliability of both the Udemy Java course and the YouTube Java tutorial videos depend on the student. Chris Ortiz's team meetings are constrained in time but offer direct project advice.

Examples of possible Java resources from Udemy and YouTube:

- Udemy course: https://www.udemy.com/course/java-for-absolute-beginners-c/
- YouTube video: https://www.youtube.com/watch?v=xTtL8E4LzTQ

Risk Assessment

Table 2: Risk assessment for learning Java

Risk	Impact	Mitigation
Team members do not take the time to learn the basic rules and foundations of Java.	Team productivity slows, which is reflected in the project's outputs and slows overall progress.	The team member will need to learn the foundations of Java through online resources such as Udemy and YouTube.
A team member is unable to attend Chris Ortiz's walkthroughs of source code changes.	The team will not be on the same page, resulting in less cohesion and a weaker understanding of the goals.	Teammates will help others catch up on the current workflow by sharing notes and through discussion.
A team member does not have an environment set up to complete changes or follow walkthroughs with the team.	That team member will not be able to contribute much to the team's work.	Team members will immediately help the teammate set up the VS Code environment and connect it to Docker.

Validation Plan

The team will export a sample TLA+ module before making a change to establish a baseline. After updating the Java source code, we will run the relevant tests and export the same module again. Every team member will do this, as everyone is expected to contribute to the source code. We will compare the outputs to confirm that the intended changes appear and that no unexpected formatting problems were introduced. Finally, we will push the changes to the main repository as part of completing the project.

Challenge Two: Accounting for All Possible Syntax Combinations and Edge Cases

Challenge Description

Parsing TLA+ syntax to support LaTeX substitution is a nontrivial task. Modern LaTeX supports rendering complex discrete math and combinatorics equations, but it requires precise syntax and argument ordering.

Success Criteria

- The output document correctly formats all operators.
- The tool has safe fallback behavior if it is unable to parse a specific section.
- The tool passes all relevant tests.

Candidate Approaches

- **Counting symbols.** Count opening symbols (`(`, `<<`, `[`, `{`), their closing counterparts, and commas and other separators, and add branching logic to handle special cases.
- **Abstract syntax tree (AST).** Use the tla2sany parser to generate abstract syntax trees that map token relationships.

Evaluation and Evidence

Evaluation will depend on testing a variety of TLA+ specifications with broad coverage of the valid specification domain. Unit tests will compare generated outputs to specific targets.

Recommended Approach

The AST-based approach is the recommended candidate because it is likely to be far more robust, easier to implement, and smaller in size. Some TLA+ syntax is difficult to parse correctly with simple symbol counting. For example, in `Pair(LAMBDA u, v : u, 3)` the comma inside the LAMBDA expression is treated as an argument separator, which is incorrect. Using an AST to map the semantic relationships between tokens avoids these types of edge-case failures entirely. This approach should require very little special-case handling logic, which makes it easier to test and less bloated.

Table 3: Comparison of parsing approaches

Criterion	Counting Symbols	AST
Complexity	Medium	Medium
Compatibility	Medium	High
Durability	Low	High

Risk Assessment

Table 4: Risk assessment for syntax handling

Risk	Impact	Mitigation
Testing fails to catch critical bugs.	Failing to catch critical bugs early in development may result in major delays later in the project.	Build comprehensive and modular testing infrastructure as early as possible in development.
Failure to parse a specific section causes the tool to fail completely.	Pull requests may not be accepted if the code is not sufficiently redundant.	Implement safe fallbacks and include fallback behavior tests in the full test suite.
Integrating the tla2sany parser is more challenging than anticipated.	The project falls behind schedule.	Prototype early so the team has enough time to pivot if necessary.

Validation Plan

We will export a sample TLA+ module before making a change to establish a baseline. After updating the Java code, we will run the relevant tests and export the same module again. We will compare the outputs to confirm that the intended changes appear and that no unexpected formatting problems were introduced.

Challenge Three: Ensuring the Correctness of Math Notation

Challenge Description

We must ensure the mathematical correctness of the LaTeX output generated by TLATeX. The tool must render each mathematical function correctly, following modern styling. Achieving this will involve checking the tool's input, its output, or both. Without validating correctness, the results could be misleading or incorrect. In this context, mathematical correctness means the rendered equation is structurally and semantically equivalent to the source expression: no symbols are dropped or altered, and grouping and precedence are unchanged. The scope is limited to the correctness of the mathematical notation and the alignment of the output with the specification. Verifying the correctness of the specification itself is out of scope.

Success Criteria

- The output document is mathematically correct.
- The line spacing, column alignment, and font size of the mathematical formulas are correct.
- The tool reports an error when a function is incorrectly formatted or does not follow the required format.
- Formatted math integrates cleanly into SanDisk documentation.

Candidate Approaches

- **Golden-file comparison.** Collect papers whose authors formatted the equations manually with LaTeX. Build tests that run the full pipeline on the specifications and compare the results to the human-created reference.
- **Input validation.** Run a parser or linter on the input that checks for balanced braces, recognized commands, and valid subscripts and superscripts.
- **Semantic validation.** Convert the TLATeX output into a form that can be compared directly with the source equation, to confirm the two are equivalent.

Evaluation and Evidence

Golden-file comparison uses a known-good reference file as the basis for tests. Because LaTeX is widely used in academia, there are many well-designed LaTeX documents against which we can check the mathematical notation in TLATeX's output. Input validation runs a parser or linter on the input to catch errors before it is passed to TLATeX. This would help ensure mathematical correctness, but it does little to check whether the output follows a formatting template. Semantic validation parses both TLATeX's output and the original TLA+ equation and compares them to confirm they are semantically equivalent. It would give the strongest assurance of mathematical correctness, but it would not help evaluate formatting.

Recommended Approach

We have chosen golden-file comparison because TLATeX output can be compared directly to known-good LaTeX examples. Input validation may catch bugs early, and semantic validation may give stronger guarantees about mathematical accuracy, but our priority is that TLATeX produces correctly

formatted math that matches manually written LaTeX. Golden-file comparison is the only candidate that measures this directly.

Table 5: Comparison of math-correctness approaches

Criterion	Golden-File Comparison	Input Validation	Semantic Validation
Output is mathematically correct	Medium	Medium	High
Typesetting accuracy	High	Medium	Medium
Errors when conditions fail	Medium	High	Medium
Template formatting	High	Low	Medium

Risk Assessment

Table 6: Risk assessment for math correctness

Risk	Impact	Mitigation
Reference quality	Tests are only as good as their references.	Select papers for the tests manually.
Limited coverage	Tests can only cover explicitly included specifications.	Make sure all necessary specifications are included.
False failures	Correct outputs can still differ from the reference without normalization.	Compare structure rather than exact text.
Error reporting	Golden-file comparison cannot detect incorrect input without examples.	Include intentionally malformed tests.
Source availability	There may be few papers that use TLA+ to generate LaTeX.	Gather evidence from LaTeX papers that use TLA+ as a source.

Validation Plan

We will find academic papers that include TLA+ specifications and corresponding LaTeX-formatted math notation, and extract the LaTeX from these papers to create golden-file comparison tests. These tests will compare the output of the TLATeX tool against the correctly formatted LaTeX.

Challenge Four: Supporting Custom Colors

Challenge Description

One challenge is how to accurately present exact hex colors when TLA+ is converted into documentation. For this project to succeed, it is vital that we accurately present color choices that support SanDisk's documentation formatting. The challenge is carrying the selected colors through TLATeX's Java code into the generated LaTeX without changing the meaning, spacing, or readability of the specification.

Success Criteria

- The source file accepts valid six-digit hex colors and identifies invalid values.
- The requested colors and their values are preserved in the generated output.
- Colors are applied only to the intended content and nothing else.
- Output exports successfully through the existing VS Code workflow and is then pushed to the main repository shared with Chris Ortiz.
- Existing spacing and math formatting are preserved, and no other formatting in the specification is disrupted.

Candidate Approaches

- **Color configuration file.** Modify the Java source code to read hex colors from a separate settings file and apply them to defined content categories. This allows the same color settings to be reused across documents, but it requires reading and validating settings and connecting them to the output generation code.
- **Color instructions in comments.** Modify the Java source code to recognize specially formatted TLA+ comments that specify colors for selected content. This would provide control over individual sections, but it requires defining the comment format and determining where each color starts.
- **Hard-coded SanDisk colors.** Hard-code SanDisk's required documentation colors into the Java source code. Because SanDisk uses the same documentation standards everywhere, hard-coding the standard hex colors means each document does not need to be configured again. This would give SanDisk a definite part of the source code that clearly defines strict rules for every document produced.

Evaluation and Evidence

The xcolor documentation confirms that LaTeX can use hexadecimal color definitions. However, this alone does not establish how custom colors should be handled in TLATeX. This repository covers the xcolor material in more depth: https://github.com/osjerick/xcolor-material.

For the configuration approach, we would need to demonstrate that the parser can read a color setting and apply it to the intended content. For the comment approach, we plan to demonstrate that the parser can recognize a color instruction and apply it to the intended content. The last approach, hard-coding SanDisk's desired color formatting into the main source code, would be the most direct but too inflexible for broader use.

Recommended Approach

The first option, a configuration file, is the best fit for our project. This approach gives us flexibility within our repository to make changes as the project progresses. We will build more complex tests as time and future updates allow.

Table 7: Comparison of custom-color approaches

Criterion	Color Configuration File	Color Instructions in Comments	Hard-Coded Colors in Source
Color control	Shared rules for content categories	Instructions for selected content	Direct and hard-coded; SanDisk keeps the same format
Reuse across documents	Reuse the same settings file	Include instructions in each document	For SanDisk only
Changes to specification files	No color comments required	Color comments required	Document-specific changes will be difficult because the code targets general documentation
Java implementation work	Read settings and apply rules	Process instructions and track their scope	Simple and direct in Java
Main risk	Settings apply to the wrong category	Color extends beyond the intended content	Per-document changes will be very difficult and will affect the entire codebase
Validation status	Not yet tested	Not yet tested	Not yet tested

Risk Assessment

Table 8: Risk assessment for custom colors

Risk	Impact	Mitigation
Invalid hex values	Incorrect colors or failed export	Validate values and report clear errors.
Color affects unintended content	Other text or symbols change color	Test where each color starts and ends.
Changes disrupt existing formatting	Incorrect spacing or math output	Compare output against an unchanged baseline.
Selected approach does not meet sponsor needs	Rework and delays	Confirm the required color control before full implementation.

Validation Plan

Chris Ortiz has given us a template that reflects SanDisk's current formatting, and we will use this exact template as the center of our testing. As we apply configuration changes, we will see their effects directly on that template file. We will learn from whatever breaks or changes in the template and make the necessary corrections. Overall, the validation will directly reflect how the template is affected by changes made through the configuration file.

Challenge Five: Supporting Image Injection

Challenge Description

Converting TLA+ to LaTeX requires the TLA+ to be parsed and converted. Neither TLA+ nor PlusCal supports image links or hyperlinks; both support only ASCII and Unicode text. Currently, TLA+ comments can be parsed as commands to the tla2tex tool, and the team wants to create such a command to inject images.

Success Criteria

- An image can be added to a file with a command that carries the image information.
- Image size and placement can be adjusted with formatting commands.
- The image is visible in the output file and matches the image that was embedded in the file.

Candidate Approaches

- **Base64.** Base64 is a binary-to-text encoding scheme that converts raw binary or text data into a safe, 64-character subset of printable ASCII characters. It was considered because TLA+ code can contain ASCII characters. The user would encode the image with an external tool, and the tla2tex source code would need decoding functionality added. Java includes a built-in, officially supported Base64 decoder. The picture would need to be decoded and merged with the TeX into a PDF document.
- **Configuration file.** The team plans to create a command that is placed next to an image ID. In a configuration file, the user adds the image's file path and specifications. The image is then inserted with those specifications when the TeX document is converted to PDF.

Evaluation and Evidence

Base64. Extended TeX typesetting engines such as LuaTeX encode and decode Base64 natively. For engines such as pdfLaTeX, shell-escape commands exist and can be used to create PDF documents from a Base64 string. `\write18` is the primitive that passes commands from the LaTeX document to the operating system. Example:

```
\documentclass{article}
\begin{filecontents*}[overwrite]{sample-image.64}
iVBORw0KGgoAAAANSUhEUgAAAPgAAABdCAYAAAH/B5vAAAAAGXRFWHRTb2Z0d2FyZQBBZ......
\end{filecontents*}
\begin{document}
  \write18{base64 -d sample-image.64 > sample-image.png}
  \includegraphics[options]{sample-image.png}
\end{document}
```

Source: https://docs.aspose.com/tex/net/latex-embedded-graphics/#decoding-base64-encoded-data

Configuration file. Gooey is an open-source Python library that turns a command-line program and its configuration into a friendly graphical user interface (Figure 2). It shows that a command can be written so that a program looks in a separate file to format its output.

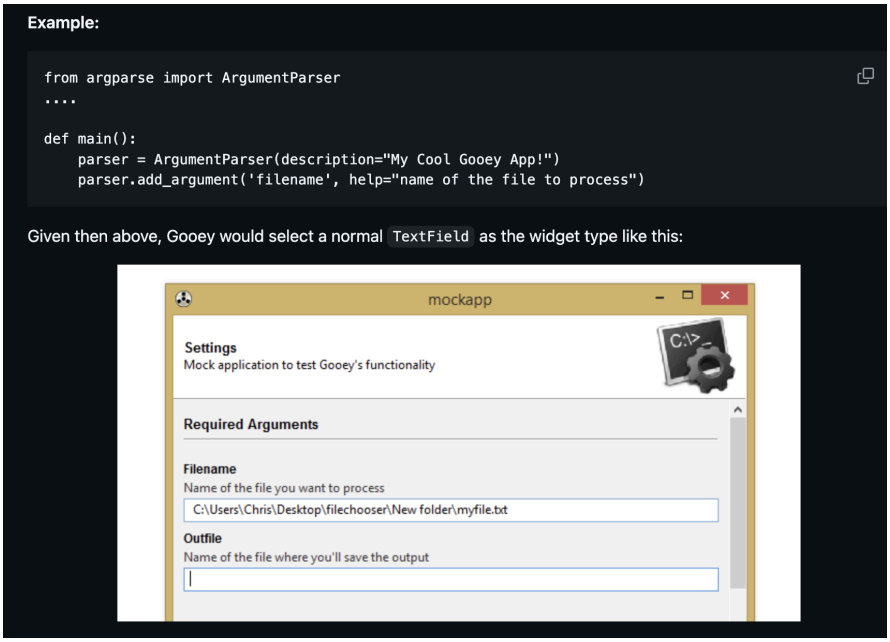


Figure 2: Gooey example (source: https://github.com/chriskiehl/gooey#examples)

Recommended Approach

The Base64 approach has the following weaknesses: the team would have to handle raw image data, so corruption is more of a risk, and testing would need to determine whether an error lies in the string or in the decoder. Its strength is that it takes less effort on the user's end: the user just inputs the Base64 string and the system handles the decoding. However, the user would have to add a comment with the file name or an image description.

The configuration file approach also has weaknesses and strengths. A weakness is that users have to configure the image path and specifications themselves. However, it is more readable and lets the user know what the image is from its path and file name. Changing the image is also more straightforward than running a new picture through an online encoder.

Table 9: Comparison of image-injection approaches

Criterion	Base64	Configuration File
Data	ASCII string	Pointer to the image
User input	Base64 string	Path to the image
Code readability	The user must remember which picture the ASCII string represents, or add a comment describing it	The user can easily tell what the picture is from its path and file name
Editing	The user must run a new image through an encoder and paste in the new string	Easy to change the path
File size	Image size increases by about 33%	File size is unchanged

Criterion	Base64	Configuration File
Image corruption	Higher chance, due to errors in handling the ASCII string	Lower chance, because raw data is not handled

Risks of the Configuration File Method

Because the team is dealing with user input, there are risks. One risk is the use of unsupported image formats. Its impact on the project is low because images can be converted to more widely supported formats. To prevent image links from breaking, the team will keep pictures in a single directory, and it is important that the team tests in a consistent environment to check for system errors. A related risk is problems with the image specifications: if a picture does not have the right specifications, the output will not be as expected. The team will then have to determine whether it is a user error or a system error, which may require pausing to review the specifications and then the software.

Table 10: Risk assessment for image injection

Risk	Impact	Mitigation
Unsupported image formats	Low	Use widely supported file formats.
Broken image link	Low	Keep pictures together in one directory while testing.
Incorrect image specifications produce unwanted output	Medium	Review the image specifications and syntax for mistakes.

Validation Plan

As in Challenge Four, the team will apply the file configurations and check whether the injection breaks the SanDisk template. The validation will check that images are inserted in the correct place and that the system follows the specifications the user provided.

Solution Integration

The main repository is a fork of the VS Code extension, used as the workspace and test harness. The code we are changing is TLATeX (tla2tex), a Java pretty-printer for TLA+. It lives in the `tlaplus/` submodule, which is our fork `tdcinaz/tlaplus` on branch `tlatex-overhaul`. Figure 3 shows how the pieces fit together.

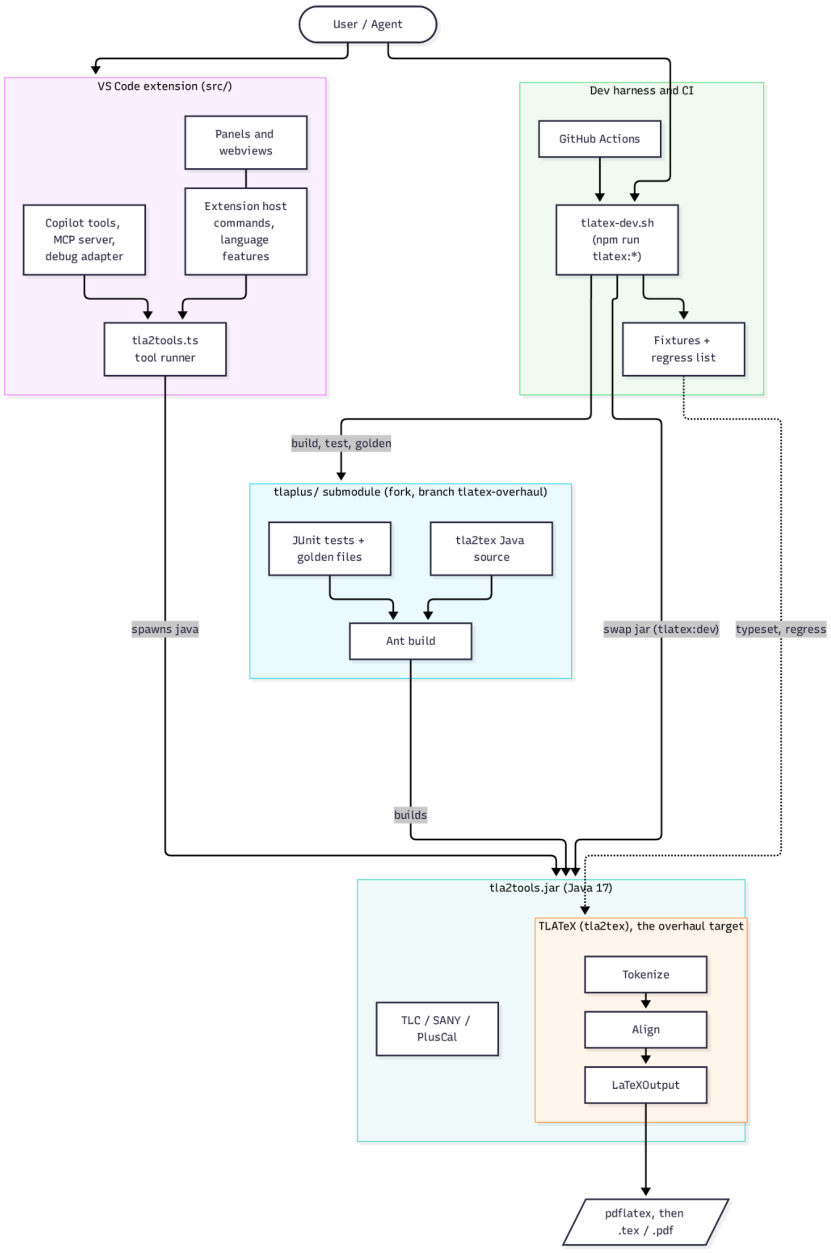


Figure 3: Solution architecture: the VS Code extension, the dev harness and CI, and the TLATeX pipeline inside tla2tools.jar

Components

- **Extension host** (`src/main.ts`, `commands/`, `completions/`, `symbols/`, `formatters/`) registers commands and language providers.
- **Tool runner** (`src/tla2tools.ts`) builds the Java options and spawns `tla2tools.jar`. TLATeX is invoked through `runTex` and `runTex2PDF`.
- **UI and AI** (`panels/`, `webview/`, `lm/`, `debugger/`) holds the webview panels, Copilot language-model tools, MCP server, and debug adapter.
- **Bundled toolchain:** `tools/tla2tools.jar`. `tlatex:dev` swaps in a local build, and that jar is never committed.
- **TLATeX pipeline** is in `tlaplus/tlatools/org.lamport.tlatools/src/tla2tex/` and has eight static stages:

    1. `BuiltInSymbols.Initialize` fills the symbol tables.
    2. `TokenizeSpec.Tokenize` splits the source into a `Token[][]`, one array per line.
    3. Proof-step and PlusCal fix-ups.
    4. Comment processing.
    5. `FindAlignments` sets the alignment information on each token.
    6. A throwaway LaTeX run measures the typeset widths.
    7. `WriteLaTeXFile` writes the final `.tex` file.
    8. `pdflatex` produces the PDF.

- **Tests:**
    - JUnit tests are in `test/tla2tex/`.
    - Input specs and their `.golden.tex` files are in `test-model/tla2tex/`.
    - Sample specs for new syntax are in `tests/fixtures/tlatex/`.
    - The release-vs-built regression list is `tests/tlatex-regress.txt`.
- **Dev harness:** `scripts/tlatex-dev.sh`, wrapped as `npm run tlatex:*`.
- **Guards:** `.githooks/` blocks committing the jar, Eclipse metadata, or a submodule pointer to an unpushed commit.
- **CI:** `.github/workflows/tlatex.yml` runs the Java build and tests.
- **Recon repository:** `scripts/recon.sh` manages a second git directory, `.git/recon.git`, that tracks `tla2tex/`, fixtures, docs, scripts, and deliverables.

Table 11: Dev harness commands

Task	Command
Rebuild the jar and swap it into the extension	`npm run tlatex:dev`
Typeset a specification	`npm run tlatex:typeset path/to/Spec.tla`
Pixel-compare two PDFs page by page	`npm run tlatex:pdfdiff before.pdf after.pdf`
Full verification before pushing	`npm run tlatex:check`
Restore the released jar	`npm run tlatex:restore`

Integrating unsupported features into the open-source code is a risk. The integrated code must be compatible with the current TLA+ language, and the TLA+ tools are written in Java. Even features added inadvertently can cause bugs later in development, so the team must test and review code before committing each change to the main branch.

The tool must produce correctly formatted LaTeX notation in order to display the more modern math symbols available in LaTeX. We will use golden-file tests that compare the TLATeX output with established LaTeX documents to implement LaTeX mathematical notation.

Conclusion

TLATeX is outdated. Unlike LaTeX, in which images can be inserted, TLATeX does not have that capability. In addition to inserting images, the client wants to be able to display characters in custom colors. The third goal is proper mathematical notation in TLATeX.

The major challenges are learning Java, accounting for all possible syntax combinations and edge cases, ensuring the correctness of math notation, and implementing custom colors and images. The team has chosen to learn Java with free video tutorials and to use an AST-based approach to account for all possible syntax combinations and edge cases. Golden-file comparison will be used to ensure the correctness of math notation. For inserting images and changing text colors, the team plans to use configuration files.

Bugs may prevent the project from succeeding. Because the software already exists, the team is concerned about latent defects and plans to proceed with caution when making modifications. The team will work on branches and test the outputs. This will assist in discovering and isolating bugs.

The project is technically feasible. Learning Java is something the team can do during the project, and the features the team plans to implement for TLA+ are features LaTeX already supports. The team's next steps are to learn Java and understand the source code so that planning for the implementation of the requirements can begin. This technical feasibility assessment should serve as a reference to guide the team's discussion of approaches.